Accenture >

Summer Internship

Final Report

Agentic AI Applications on Azure — Design, Development & Deployment

Name	Anand Raj
Institute	Kalinga Institute of Industrial Technology (KIIT)
Program	B.Tech, Computer Science and Engineering — 7th Semester
Role	Associate Software Engineer Intern (Custom Software Engineering Associate)
Location	Bengaluru, India
Duration	18 May 2026 – 10 July 2026
Project Lead	Sreeja Nair
Buddy / Mentor	Sridharan Surendran

Let there be change.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Accenture India for providing me with the opportunity to complete my summer internship and gain hands-on exposure to real-world cloud and Agentic AI application development. This internship helped me understand how modern AI-powered solutions are designed, implemented, tested and deployed using enterprise-grade tools and cloud services.

I am especially thankful to my Project Lead, **Sreeja Nair**, for her valuable guidance, support and direction throughout the internship. Her inputs helped me stay aligned with the expected outcomes of the project and encouraged me to approach each task with clarity, structure and a practical implementation mindset.

I would also like to extend my heartfelt thanks to my Buddy and Mentor, **Sridharan Surendran**, for his continuous technical assistance, feedback and encouragement. His support helped me understand the requirements of each task assigned to me , resolve implementation doubts, improve my debugging approach and document the work in a more professional manner.

I am grateful to the mentors, colleagues and team members who supported me during different stages of the internship. Their guidance and encouragement helped me build confidence.

This internship gave me a valuable learning environment where I could apply classroom knowledge to practical software engineering tasks. It also helped me improve my problem-solving ability, understand cloud-native development practices, and learn the importance of structured logging, error handling, secure configuration and clear technical documentation.

Finally, I would like to thank my institute, **Kalinga Institute of Industrial Technology (KIIT)**, for giving me the academic foundation that enabled me to take up this internship opportunity. The experience has been an important milestone in my learning journey and has strengthened my interest in cloud computing, artificial intelligence and enterprise software development.

EXECUTIVE SUMMARY

This report documents the work completed during my Summer Internship at Accenture India, where I designed, implemented and deployed four end-to-end Agentic AI applications on Microsoft Azure. Each use case was built as an HTTP-triggered Azure Function using the Node.js runtime, integrated with Azure AI Foundry (GPT-5) for language reasoning, and instrumented with Azure Application Insights for full observability.

The four use cases were intentionally structured to progress from a single-model API to a fully orchestrated multi-agent workflow. The first use case delivered a serverless summarization API. The second introduced lightweight Retrieval-Augmented Generation over a text knowledge base. The third produced schema-conforming structured outputs from unstructured meeting notes. The fourth combined multiple specialised agents — Planner, Research, Summarizer and Reviewer — under an Orchestrator to answer complex research queries.

Along the way I gained deep, hands-on exposure to serverless design, prompt engineering, structured JSON generation, retrieval-based grounding, telemetry, deployment, and real-world debugging of cloud and syntax errors. The following sections describe each use case in detail, including its objective, my learnings, implementation approach, workflow diagram, and the challenges faced along with how I resolved them.

At a Glance

Use Case	One-line Value
Use Case 1 — AI Text Summarizer API	Serverless GPT-5 powered summarization service with configurable summary length.
Use Case 2 — Intelligent FAQ Assistant	Knowledge-base grounded Q&A using keyword retrieval plus GPT-5 answering.
Use Case 3 — Meeting Notes Analyzer Agent	Structured extraction of decisions, action items, risks, blockers and dependencies.
Use Case 4 — Multi-Agent Research Assistant	Orchestrated Planner / Research / Summarizer / Reviewer agents for research queries.

TECHNOLOGY STACK

All four use cases share a common technology foundation, which allowed me to keep them in a single Azure Functions repository while cleanly separating agents, functions and data. The table below summarises the stack.

Technology / Tool	Purpose in the Project
Azure Functions	Used as the serverless compute layer for exposing all four use cases as HTTP-triggered APIs.
Node.js	Used as the runtime and implementation language for writing function logic, file handling, API processing and agent modules.
Azure AI Foundry	Used to access the GPT-5 model for summarization, question answering, meeting note analysis and agent reasoning.
GPT-5 Model	Used as the core AI reasoning engine for generating summaries, grounded answers, structured JSON outputs and reviewed research responses.
Application Insights	Used for telemetry, request monitoring, response-time tracking, failure analysis and runtime logging across the Azure Functions project.
PowerShell	Used to test local and deployed HTTP endpoints using Invoke-RestMethod and inspect JSON responses during development.
Visual Studio Code	Used as the main development environment for editing code, managing project files, running local functions and deploying to Azure.

USE CASE 1 : AI Text Summarizer

Objective

Build a serverless HTTP API using Azure Functions that accepts user-provided text and returns a concise AI-generated summary in JSON format using GPT-5 deployed via Azure AI Foundry, with response timing, structured logging, input validation and Application Insights monitoring.

What I Learnt

- Serverless application development using Azure Functions with HTTP triggers.
- Environment variable management via local.settings.json and Azure App Settings.
- Deploying and consuming GPT-5 models through Azure AI Foundry.
- Prompt engineering for summarization with a configurable summary length.
- Response-time measurement using Date.now() at the start and end of the call.
- Application Insights integration for logs and telemetry.
- End-to-end cloud-native deployment to an Azure Function App.

How I Implemented It

- I created an HTTP-triggered Azure Function named **TextSummarizer** using the Node.js Azure Functions programming model. The function accepts a POST request body containing the input text and a summaryLength value such as short, medium, or detailed.
- I added request validation so that empty text or unsupported summary length values return a clear 400 response instead of causing runtime errors. This made the API more reliable and user-friendly during testing.
- I connected the function to Azure AI Foundry by reading **FOUNDRY_ENDPOINT**, **FOUNDRY_API_KEY**, and **FOUNDRY_MODEL** from environment variables in local.settings.json and Azure App Settings, avoiding hardcoded credentials in the code.
- I designed a length-aware prompt where short summaries return a few sentences, medium summaries provide more context, and detailed summaries generate a fuller paragraph. The GPT-5 response was extracted from the model output and returned as structured JSON.
- I measured response time using Date.now(), logged important execution stages with context.log(), deployed the function to Azure, and connected it to Application Insights to monitor server requests, latency, and failures.

Mechanism Flow Chart

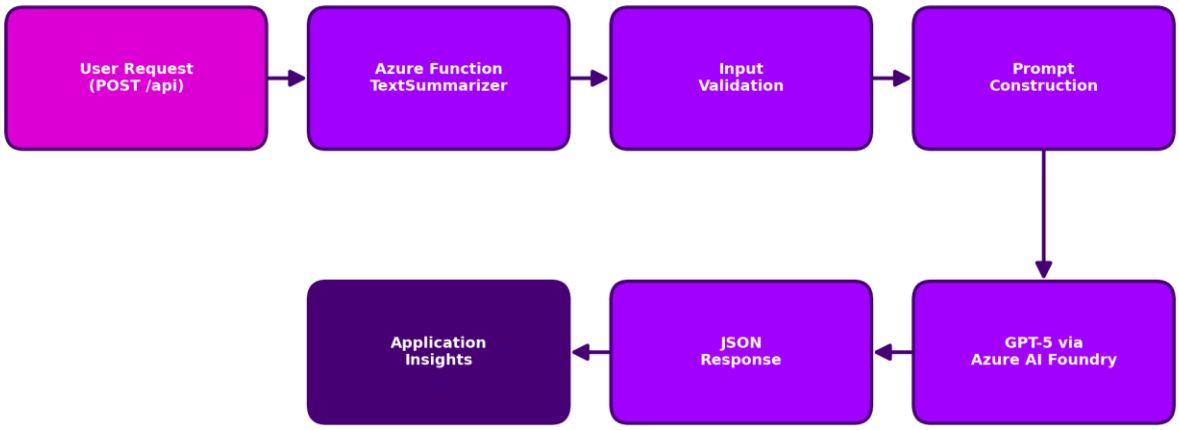

Figure 1 — AI Text Summarizer API: End-to-End Workflow

Challenges Faced and How I Handled Them

Challenge	Resolution
Understanding the role of Azure Function configuration files (host.json, package.json, local.settings.json).	Studied each file's responsibility inside the Azure Functions runtime and referenced official Microsoft documentation.
GPT-5 integration failing with authentication errors.	Verified endpoint URL, API key and deployment name in Azure AI Foundry and corrected the environment variables.
Environment variables not loading during local runs.	Corrected local.settings.json format and restarted the Functions host to reload settings.
Response payload was hard to read in PowerShell.	Piped output through ConvertTo-Json -Depth 10 for clean, indented output during testing.

USE CASE 2 : Intelligent FAQ Assistant

Objective

Build an HTTP-triggered Azure Function that answers user questions using a predefined knowledge-base document via a lightweight Retrieval-Augmented Generation flow — extracting the most relevant text chunk from the document and passing it as grounded context to Azure AI Foundry GPT-5, while logging telemetry through Application Insights.

What I Learnt

- Creating a new HTTP function inside an existing Function App project with shared configuration.
- Using a text document as a knowledge source instead of exact Q/A pairs.
- Why exact full-question matching (indexOf on the full question) is weak for natural language.
- Keyword extraction with stop-word filtering for better retrieval.
- Prompt engineering to strictly ground the model in the provided context.
- Handling ENOENT and configuration errors gracefully.
- Connecting Application Insights using APPLICATIONINSIGHTS_CONNECTION_STRING.

How I Implemented It

- I created **FAQAssistant.js** as an HTTP-triggered Azure Function that receives a user question through a POST request and validates that the question field is present before processing.
- I stored the knowledge content in **src/data/knowledgebase.txt** and read it using fs.readFileSync with path.join(process.cwd(), 'src', 'data', 'knowledgebase.txt') so the file path works reliably during local execution.
- I initially tried full-question matching, but it failed for natural language questions. To improve retrieval, I implemented keyword extraction by lowercasing the question, removing punctuation, tokenising it, and filtering common stop words.
- I selected a relevant context window around the first matched keyword and passed that context with the original question to GPT-5 using a strict grounding prompt, so the model answered only from the provided knowledge base.
- I returned a clean JSON response containing question, answer, source, and responseTimeMs, while using context.log() and Application Insights to track request execution, retrieval success, and possible failures.

Mechanism Flow Chart

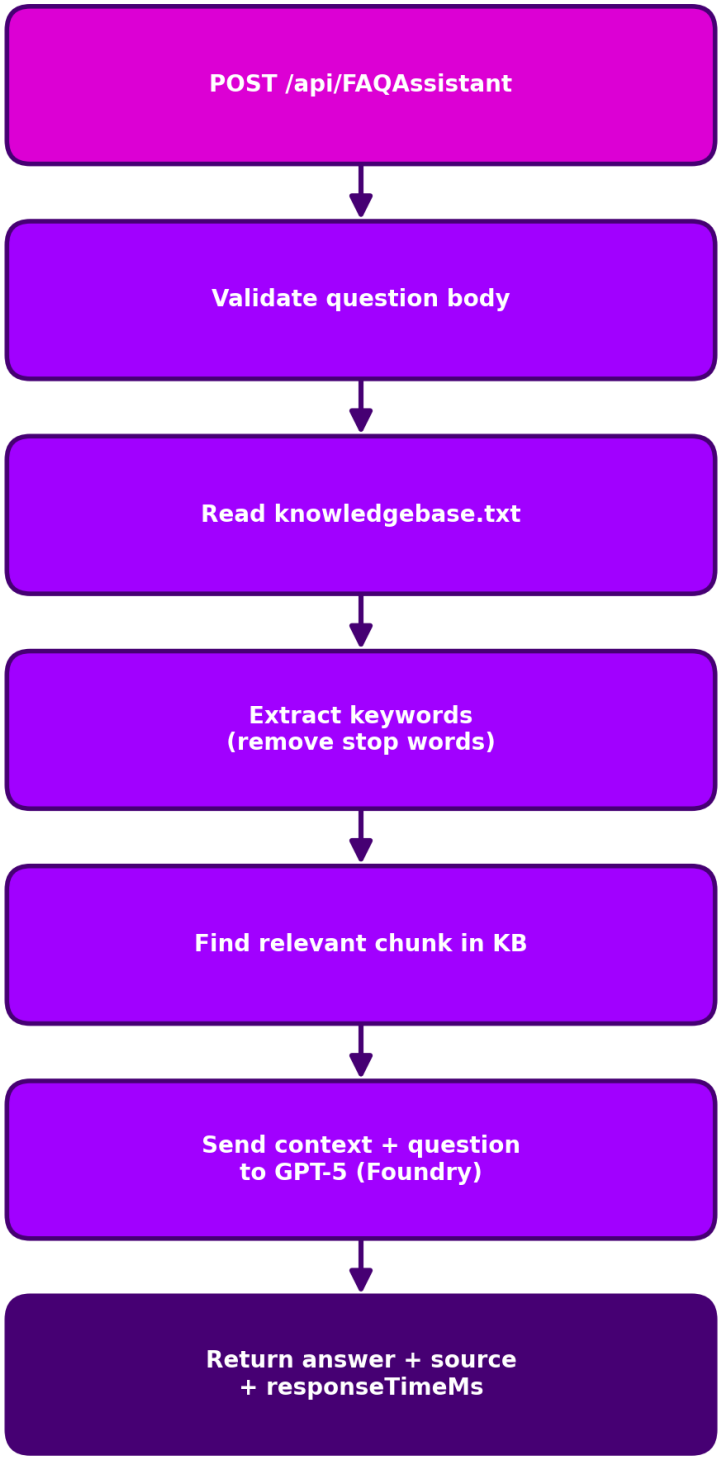

Figure 2 — Intelligent FAQ Assistant: Request Processing Flow

Challenges Faced and How I Handled Them

Challenge	Resolution
ENOENT error — knowledgebase.txt not found at runtime.	Corrected the file path from the project root to src/data/knowledgebase.txt using path.join(process.cwd(), 'src', 'data', 'knowledgebase.txt').
Exact-match search failed for natural questions like 'What is Microsoft Azure?'	Implemented keyword extraction with stop-word filtering so retrieval matches meaningful terms in the KB.
Application Insights showed zero metrics initially.	Added APPLICATIONINSIGHTS_CONNECTION_STRING to local.settings.json and restarted the host; Server Requests started appearing.
Response payload was too crowded due to embedded relevantContent.	Removed relevantContent from the final response, keeping only question, answer, source and responseTimeMs.
Model sometimes answered from general knowledge rather than the KB.	Rewrote the prompt to strictly restrict answers to the provided KB context, with a fallback message otherwise.

USE CASE 3 : Meeting Notes Analyzer Agent

Objective

Build an AI-powered Azure Function that processes raw meeting notes and transforms unstructured discussion text into structured, actionable insights — a concise summary, action items with owners, risks, blockers and dependencies — using Azure AI Foundry GPT-5, and persist the result as a JSON artefact for downstream consumption.

What I Learnt

- Designing AI-powered workflows that produce structured JSON outputs.
- Prompt engineering for schema-conforming responses.
- JSON parsing, validation and error recovery for LLM outputs.
- File system operations in Node.js (reading input, writing analysis-result.json).
- Capturing logs and telemetry using context.log().
- Robust error handling with try/catch across file, model and parsing stages.

How I Implemented It

- I created **MeetingNotesAnalyzer.js** as an HTTP-triggered Azure Function that reads raw meeting notes from **src/data/meetingnotes.txt** and treats them as the input for AI-based analysis.
- I added configuration validation before the model call to ensure the Azure AI Foundry endpoint, API key, and model deployment name are available in the environment before execution continues.
- I designed a structured prompt instructing GPT-5 to return only a valid JSON object containing summary, actionItems, risks, blockers, and dependencies, making the output suitable for downstream consumption.
- I parsed the model response using JSON.parse inside a try/catch block so malformed JSON responses could be caught and handled clearly instead of breaking the entire function without explanation.
- I saved the final structured output into **analysis-result.json** and also returned it through the API response with source and responseTimeMs, while using context.log() and Application Insights to debug errors and monitor execution.

Mechanism Flow Chart

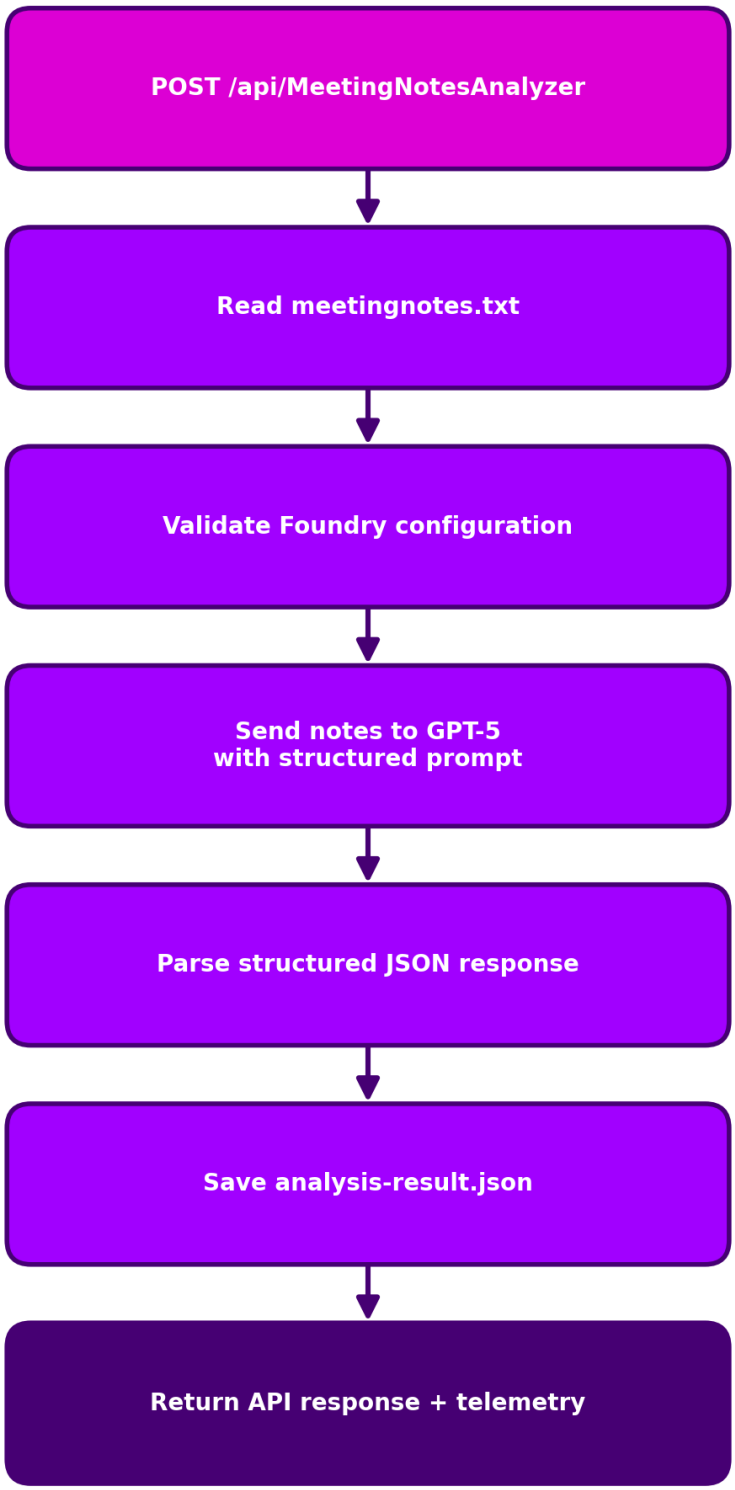

Figure 3 — Meeting Notes Analyzer: Structured Extraction Flow

Challenges Faced and How I Handled Them

Challenge	Resolution
Repeated 500 Internal Server Error during initial testing.	Traced errors via func start logs, identified misconfigured Foundry endpoint and JSON parse failures, and fixed the prompt plus environment variables.
Model occasionally returned malformed JSON.	Strengthened the prompt to enforce strict JSON structure and wrapped JSON.parse in try/catch with a clear error message.
File path issues on Windows during file read/write.	Used path.join with process.cwd() to construct portable, absolute paths for input and output files.
Extracting the correct text field from the Foundry response object.	Standardised on aiResult.output[1].content[0].text after inspecting the response schema carefully.
Ensuring each action item had owner and task fields.	Refined the prompt with an explicit example schema so the model produced consistent action item objects.

USE CASE 4 : Multi-Agent Research Assistant

Objective

Build an agentic AI application where multiple specialised agents (Planner, Research, Summarizer, Reviewer) collaborate under an Orchestrator to process a complex research query end-to-end, exposed through a single Azure Function endpoint with Application Insights observability.

What I Learnt

- Difference between single-agent and multi-agent AI applications.
- Separating responsibilities across agents for explainability and modularity.
- Orchestrator pattern for coordinating multi-step agent workflows.
- Passing structured outputs from one agent to another.
- Integrating GPT-5 selectively in agents that require reasoning.
- Using environment variables for Foundry configuration.
- Using Azure Functions as an HTTP API layer for agentic AI workloads.
- Debugging syntax errors using Azure Functions stack traces.
- The importance of output formatting (Markdown structure) in AI responses.

How I Implemented It

- I created a dedicated **src/agents** folder containing Planner, Research, Summarizer, Reviewer, and Orchestrator agent files so that each part of the workflow had a clear responsibility.
- I added **MultiAgentResearchAssistant.js** as the HTTP-triggered entry point. It accepts a POST request with a query field, validates the input, measures response time, and then passes the query to the Orchestrator Agent.
- The Orchestrator Agent coordinates the full sequence: Planner creates the task plan, Research reads context from researchData.txt, Summarizer calls GPT-5 to generate the core response, and Reviewer improves the final formatting and clarity.
- I used Azure AI Foundry configuration through environment variables so the Summarizer and Reviewer agents could call GPT-5 securely without hardcoding endpoint, API key, or model deployment details.
- I logged every major stage with context.log() and connected the workflow to Application Insights, which helped track server requests, response time, execution flow, and failures across the complete multi-agent pipeline.

Mechanism Flow Chart

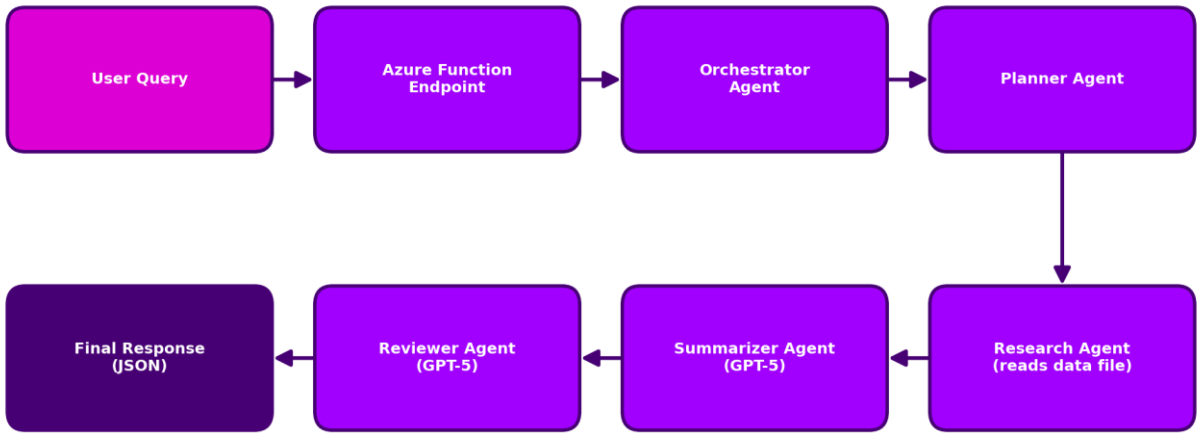

Figure 4 — Multi-Agent Research Assistant: Orchestrated Workflow

Challenges Faced and How I Handled Them

Challenge	Resolution
Deciding whether to create a new Azure Functions project or extend the existing one.	Chose to continue in the same repository so all internship use cases live together, adding dedicated agents/ and functions/ subfolders.
Understanding the true role of an Orchestrator.	Clarified that a dedicated Orchestrator makes the design explicitly agentic — it holds the workflow logic and cleanly coordinates Planner, Research, Summarizer and Reviewer agents.
'Invalid shorthand property initializer' syntax error in plannerAgent.js.	Identified the issue in the stack trace — an accidental '=' instead of ':' in an object literal — and corrected it to tasks: [...].
Final answer displayed as a crowded JSON string in PowerShell.	Improved the Reviewer Agent prompt to produce Markdown-formatted output with headings, bullets and spacing.
Application Insights resource name was too narrow (text-summarizer-insights).	Reviewed the naming and moved towards a project-scoped name such as AI-Internship-Insights.
Extracting the generated text from Foundry model responses.	Standardised on aiResult.output[1].content[0].text across all agents that call the model.

APPLICATION INSIGHTS INTEGRATION & TELEMETRY MONITORING

After completing all four Azure Function use cases, I integrated Azure Application Insights as a common monitoring layer for the complete project. This helped me observe how the APIs behaved after execution instead of depending only on terminal logs. With Application Insights, I could track request count, execution time, failures, response trends and runtime logs from one central place.

How I Integrated Application Insights with Azure Functions

- I connected the Azure Function App with an Application Insights resource so that all four HTTP-triggered APIs could send telemetry to the same monitoring workspace.
- I added the **APPLICATIONINSIGHTS_CONNECTION_STRING** value in local.settings.json for local testing and also configured the same setting in the deployed Function App configuration in Azure.
- I used **context.log()** inside the functions and agents to record key execution checkpoints such as request received, input validation completed, GPT-5 call started, response received, JSON parsed and final response returned.
- I tested the functions using PowerShell and verified that the Application Insights dashboard started showing server requests, response time trends and failure details after the API calls were executed.
- This monitoring setup helped me debug issues such as missing environment variables, authentication failures, malformed JSON outputs, file path issues and 500 Internal Server Errors more efficiently.

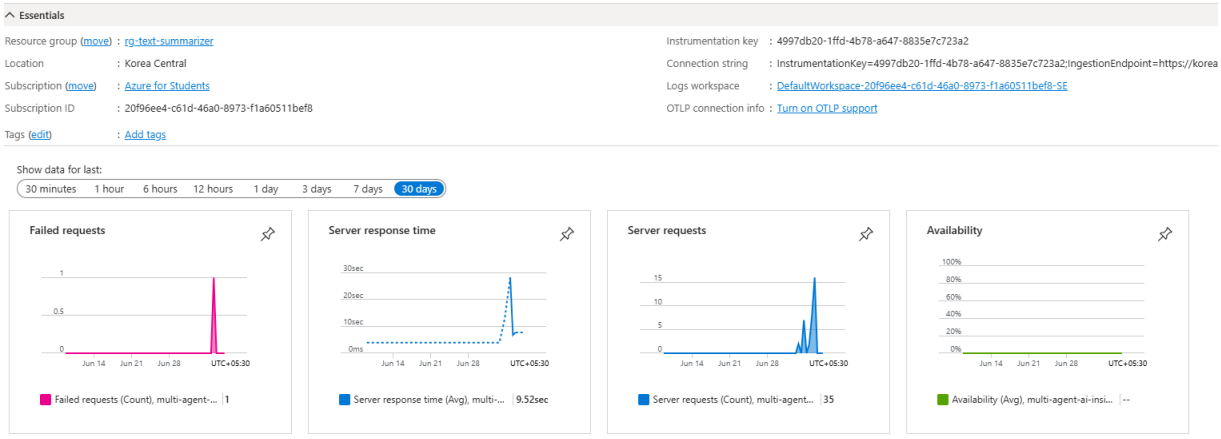

Figure 5 — Application Insights telemetry view showing server requests, response duration and monitoring trends

Explanation of the Graphs

- **Server Requests:** The request graph confirms that the deployed Azure Function endpoints were being called successfully. Each API test triggered from PowerShell or the deployed URL contributed to the request count.
- **Response Time / Duration:** The duration trend helped me understand how long the functions took to complete, especially for GPT-5 based requests where model response time affects the total execution time.
- **Failures:** Failure indicators helped identify unsuccessful executions such as 500 Internal Server Errors, missing environment variables, authentication issues or malformed model responses.
- **Timeline Spikes:** The spikes in the graph matched the periods when I repeatedly tested the APIs, which confirmed that telemetry was being captured correctly in near real time.
- **Operational Value:** These graphs made the project more production-ready because I could monitor health, performance and reliability across all four use cases from one dashboard.

PROJECT FILE STRUCTURE

All four use cases live inside a single Azure Functions project, with clean separation between agents, functions and data. This structure kept the codebase modular while allowing shared configuration (host.json, local.settings.json, package.json) across every use case.

```
AI_INTERNSHIP_PROJECT/
│
├── src/
│   ├── agents/
│   │   ├── orchestratorAgent.js
│   │   ├── plannerAgent.js
│   │   ├── researchAgent.js
│   │   ├── summarizerAgent.js
│   │   └── reviewerAgent.js
│   │
│   ├── data/
│   │   ├── knowledgebase.txt
│   │   ├── meetingnotes.txt
│   │   └── researchData.txt
│   │
│   └── functions/
│       ├── TextSummarizer.js
│       ├── FAQAssistant.js
│       ├── MeetingNotesAnalyzer.js
│       └── MultiAgentResearchAssistant.js
│
├── host.json
├── local.settings.json
├── package.json
└── package-lock.json
```

The data folder isolates all knowledge sources used by the retrieval and analyzer functions. The agents folder contains the reusable, single-responsibility agent modules used by the Multi-Agent Research Assistant. The functions folder hosts the four HTTP-triggered Azure Functions — one per use case — that expose the public API surface of the project.

OVERALL LEARNING & REFLECTION

Working across these four use cases gave me a natural progression from a single-model API to a fully orchestrated multi-agent AI workflow. The first use case grounded me in the fundamentals of Azure Functions, environment configuration and Azure AI Foundry integration. The second taught me why exact matching is not sufficient in real-world QA scenarios and how a small amount of retrieval logic (keyword extraction plus a context window) can dramatically improve LLM answers when combined with strict prompt grounding.

The third use case moved me from free-text output to schema-conforming structured JSON generation, which is a foundational skill for building enterprise AI systems where downstream services consume the output. Debugging repeated 500 errors taught me to read Azure Functions logs carefully, validate model output defensively, and wrap fragile operations in try/catch. The fourth use case pulled everything together — I built a Planner, Research, Summarizer and Reviewer, and orchestrated them explicitly, which finally made the 'agentic' pattern click conceptually.

Beyond the technical skills, I grew significantly in independent problem-solving. Fixing PowerShell execution policy issues, tracking down malformed JSON, correcting file-path bugs, and iterating on prompts until the output was truly production-ready — all of these are practical, cloud-native software engineering habits I take away from this internship.

Along with the technical implementation work, I also had the opportunity to stay involved as a silent observer in a real industry-level project environment. By attending team calls and observing regular discussions, I gained an understanding of how project work is carried out in a professional setting, how responsibilities are shared across team members, how updates are communicated, and how collaboration, coordination and team effort contribute to successful delivery. This exposure helped me look beyond individual coding tasks and understand the importance of teamwork, communication, accountability and structured execution in enterprise software projects.

CONCLUSION

All four use cases were successfully designed, implemented, tested locally with PowerShell and Invoke-RestMethod, and deployed as Azure Functions integrated with Azure AI Foundry (GPT-5) and Application Insights. Each solution meets its stated objective and demonstrates a practical, real-world application of agentic AI patterns on Azure.

The internship strengthened my cloud-native, AI and software engineering skills and provided a strong foundation for future work on agentic AI systems. The consolidated repository, structured logging and Application Insights coverage make the solution suitable as a reference implementation for future enterprise AI projects.